Assignment 2 Search Engine using Hadoop MapReduce

1. Methodology

1.1 System Overview

This project implements a simple search engine that indexes a corpus of plain text documents and retrieves the most relevant ones for a given user query. The ranking is based on the BM25 scoring model. The system consists of three main components:

1. **Data preparation** -- transforms raw text files into a structured format suitable for batch processing.
2. **Indexer** -- two Hadoop MapReduce pipelines that build an inverted index and a vocabulary with document frequency statistics. The results are stored in Cassandra.
3. **Ranker** -- a PySpark application that reads the index from Cassandra, computes BM25 scores using RDD operations, and returns the top 10 documents.

1.2 Infrastructure

The system runs inside Docker containers orchestrated with `docker-compose.yml`:

Container	Image	Role
`cluster-master`	`firasj/spark-docker-cluster`	HDFS NameNode, YARN ResourceManager, Spark driver, runs all scripts
`cluster-slave-1`	`firasj/spark-docker-cluster`	HDFS DataNode, YARN NodeManager, executes Spark/MapReduce tasks
`cassandra-server`	`cassandra`	Single-node Cassandra cluster for storing index data

The `app/` folder is mounted into `cluster-master` at `/app`. The entrypoint `app.sh` runs the entire pipeline from start to finish: it starts Hadoop/YARN services, installs Python dependencies, prepares data, builds the index, stores it in Cassandra, and runs demonstration queries.

1.3 Data Preparation

The corpus consists of 100 Wikipedia articles stored as plain text files in `app/data/`. Each file follows the naming convention `<doc_id>_<doc_title>.txt`, for example `143425_A_Day_in_the_Life.txt`.

The preparation step has two parts:

`prepare_data.sh`: uploads the local `data/` directory to HDFS at `/data/`, then launches the PySpark script.

`prepare_data.py`: uses `sc.wholeTextFiles()` to read all `.txt` files into an RDD of `(filepath, content)` pairs. A `parse_doc` function extracts the document ID and title from the filename (splitting on the first underscore, replacing remaining underscores with spaces) and cleans the content by replacing tabs and newlines with spaces. The resulting RDD is coalesced into a single partition and saved to HDFS at `/input/data` as tab-separated lines:

```
<doc_id>\t<doc_title>\t<doc_text>
```

This single-partition file serves as input to the first MapReduce pipeline.

1.4 Indexer Design

The indexer uses two Hadoop Streaming MapReduce pipelines. This separation keeps each pipeline focused on a single task, which makes debugging easier and produces cleanly separated output.

Pipeline 1: Inverted Index (`mapper1.py` / `reducer1.py`)

Purpose: build an inverted index that maps each term to the documents it appears in, along with the term frequency (tf) and document length (dl).

Mapper (`mapper1.py`):

- Reads each input line (one document per line) and splits it into `doc_id`, `doc_title`, and `doc_text` by tab.
- Tokenizes the document text: converts to lowercase, removes all non-alphanumeric characters except spaces, and splits on whitespace.
- Counts the frequency of each unique term within the document and records the total document length.
- Emits one line per unique (term, document) pair in the format: `term\tdoc_id\tdoc_title\ttf\tdl`.

Since each input line contains one entire document, and Hadoop assigns each line to exactly one mapper, every (term, doc_id) pair is emitted exactly once across all mappers. This means no aggregation is needed in the reducer.

Reducer (`reducer1.py`): identity reducer that passes all records through unchanged. The data is already complete from the mapper.

Output: stored in HDFS at `/indexer/index`.

Pipeline 2: Vocabulary with Document Frequencies (`mapper2.py` / `reducer2.py`)

Purpose: compute the document frequency (df) for each term -- the number of distinct documents containing that term.

Mapper (`mapper2.py`): reads each line from the Pipeline 1 output and emits `term\t1`. Since each line in the index represents one unique (term, doc) pair, emitting 1 for each line and summing per term gives the correct df.

Reducer (`reducer2.py`): standard word-count pattern. Accumulates counts for consecutive lines with the same key (term), outputs `term\tdf` when the key changes.

Output: stored in HDFS at `/indexer/vocab`.

Both pipelines are executed by `create_index.sh`, which accepts an optional argument for the input path (defaults to `/input/data`).

1.5 Cassandra Storage

The script `store_index.sh` calls `app.py`, which:

1. Waits for Cassandra to become available (retry loop with 10-second intervals, up to 30 attempts).
2. Creates the keyspace `search_engine` with `SimpleStrategy` and replication factor 1.
3. Creates four tables.
4. Reads index and vocabulary data from HDFS using PySpark.
5. Inserts data into Cassandra using prepared statements via `cassandra-driver`.

Cassandra Schema

Keyspace: `search_engine`

Table	Columns	Primary Key	Purpose
`inverted_index`	`term` text, `doc_id` text, `doc_title` text, `tf` int, `dl` int	`(term, doc_id)`	Posting lists: for a given term, all documents containing it with tf and dl

Table	Columns	Primary Key	Purpose
`vocabulary`	`term` text, `df` int	`(term)`	Document frequency for each term in the corpus
`doc_stats`	`doc_id` text, `doc_title` text, `dl` int	`(doc_id)`	Per-document metadata: title and length
`corpus_stats`	`id` int, `num_docs` int, `avgdl` double	`(id)`	Corpus-level statistics: total document count and average document length

Design rationale: the `inverted_index` table uses a compound primary key `(term, doc_id)`. This means all posting entries for a single term are stored in the same Cassandra partition, which makes lookups by term very efficient -- exactly the access pattern needed during query answering. The `vocabulary` table is partitioned by `term` so that the df of any query term can be fetched with a single-partition read. The `doc_stats` and `corpus_stats` tables store precomputed values so that the ranker does not need to recompute them at query time.

The `doc_stats` table is derived from the inverted index during the loading step: for each unique `doc_id` encountered in the index data, the title and document length are extracted and inserted. The corpus-level statistics (N and avgdl) are computed as the number of unique documents and the mean of their lengths.

1.6 BM25 Ranker

The ranker is implemented in `query.py`, a PySpark application submitted to YARN via `search.sh`.

BM25 Formula

For a query *q* consisting of terms *t*, the BM25 score for a document *d* is:

```
BM25(q, d) = sum over t in q of: log(N / df(t)) * ((k1 + 1) * tf(t,d)) / (k1 * ((1
- b) + b * dl(d) / avgdl) + tf(t,d))
```

Where:

- N = total number of documents in the corpus
- df(t) = number of documents containing term t
- tf(t, d) = frequency of term t in document d
- dl(d) = length of document d (in tokens)

- avgdl = average document length across the corpus
- k1 = 1.5, b = 0.75 (standard BM25 hyperparameters)

Implementation

1. The query string is received as a command-line argument.
2. It is tokenized using the same procedure as the indexer (lowercase, remove non-alphanumeric characters, split on whitespace), and duplicate terms are removed.
3. The driver connects to Cassandra and retrieves:
   - Corpus stats (N, avgdl) from `corpus_stats`.
   - Document frequency for each query term from `vocabulary`.
   - All posting list entries for the query terms from `inverted_index`.
4. The posting list entries are parallelized into a Spark RDD. The BM25 parameters and term-df map are broadcast to all executors.
5. An RDD `.map()` operation computes the per-(term, document) BM25 contribution.
6. A `.reduceByKey()` aggregates scores across all query terms for each document.
7. A `.sortBy()` (descending) followed by `.take(10)` retrieves the top 10 results.
8. The results are printed to stdout.

The `search.sh` script runs this application on YARN:

```
spark-submit --master yarn --archives /app/.venv.tar.gz#.venv query.py "$1"
```

The virtual environment is distributed to executors via `--archives` so that dependencies like `cassandra-driver` are available on worker nodes.

1.7 Tokenization

Both the indexer and the ranker use the same tokenization strategy to ensure consistency:

1. Convert text to lowercase.
2. Remove all characters that are not letters, digits, or whitespace (using regex `[^a-z0-9\s]`).
3. Split on whitespace.

This is a simple but effective approach for the assignment scope. It strips punctuation and normalizes case, which ensures that "Dogs", "dogs", and "dogs." are all treated as the same term.

1.8 Execution Flow

The complete pipeline is orchestrated by `app.sh`:

1. Start SSH and Hadoop services (`start-services.sh`).
2. Create a Python virtual environment and install dependencies (`requirements.txt`).
3. Package the virtual environment for YARN distribution (`venv-pack`).
4. Prepare data: upload to HDFS, create `/input/data` (`prepare_data.sh`).
5. Index: run MapReduce pipelines and store in Cassandra (`index.sh`).
6. Search: run BM25 queries (`search.sh`).

The instructor can reproduce the entire pipeline by running `docker compose up` from the repository root.

---

2. Demonstration

2.1 How to Run

Prerequisites: Docker and Docker Compose must be installed.

Steps:

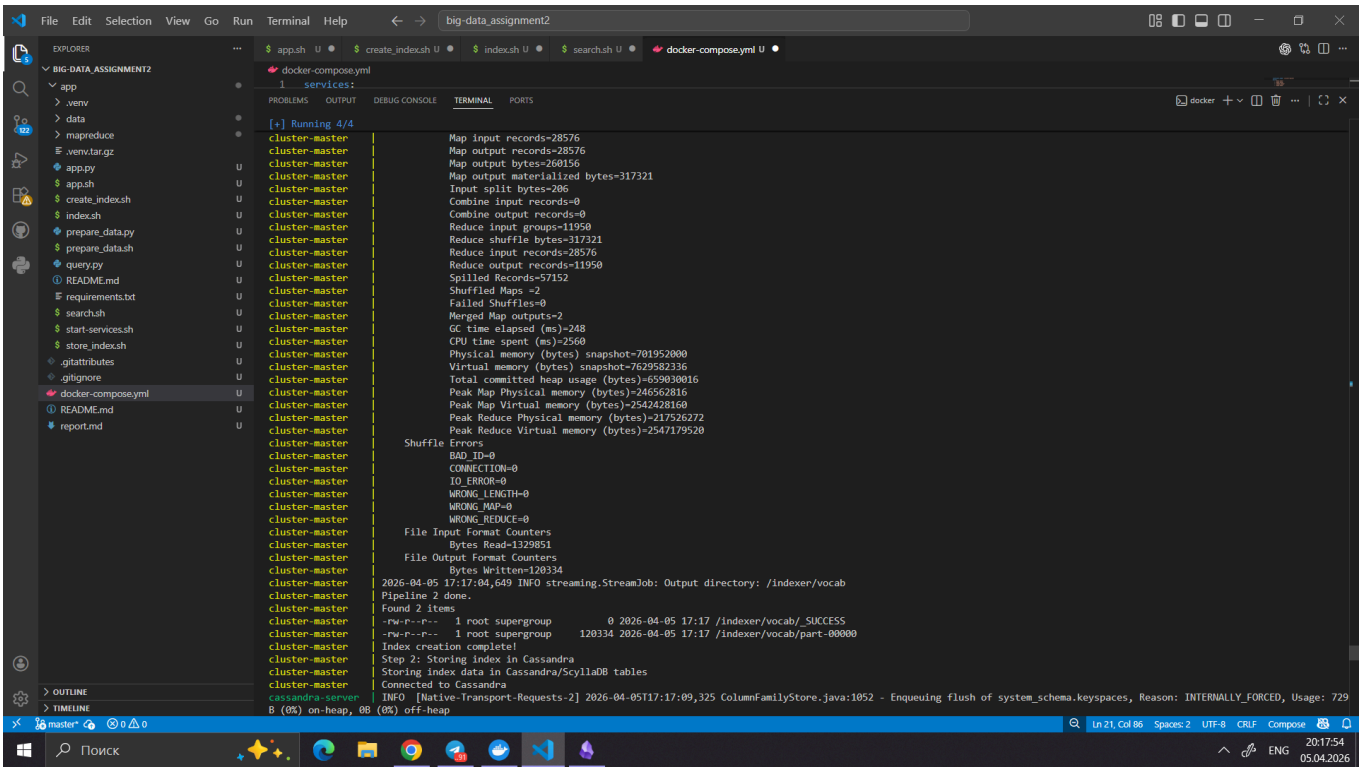
1. Clone the repository.
2. Navigate to the repository directory.
3. Run:
   ```bash

docker compose up

``` `4. The entrypoint script` app.sh` will execute the entire pipeline automatically. The output appears in the terminal.

2.2 Indexing 100 Documents

After the data preparation step completes, the MapReduce pipelines are executed. The terminal output confirms both pipelines completed successfully.



After the MapReduce pipelines finish, the index is loaded into Cassandra. The terminal displays a summary:

```
==================================================

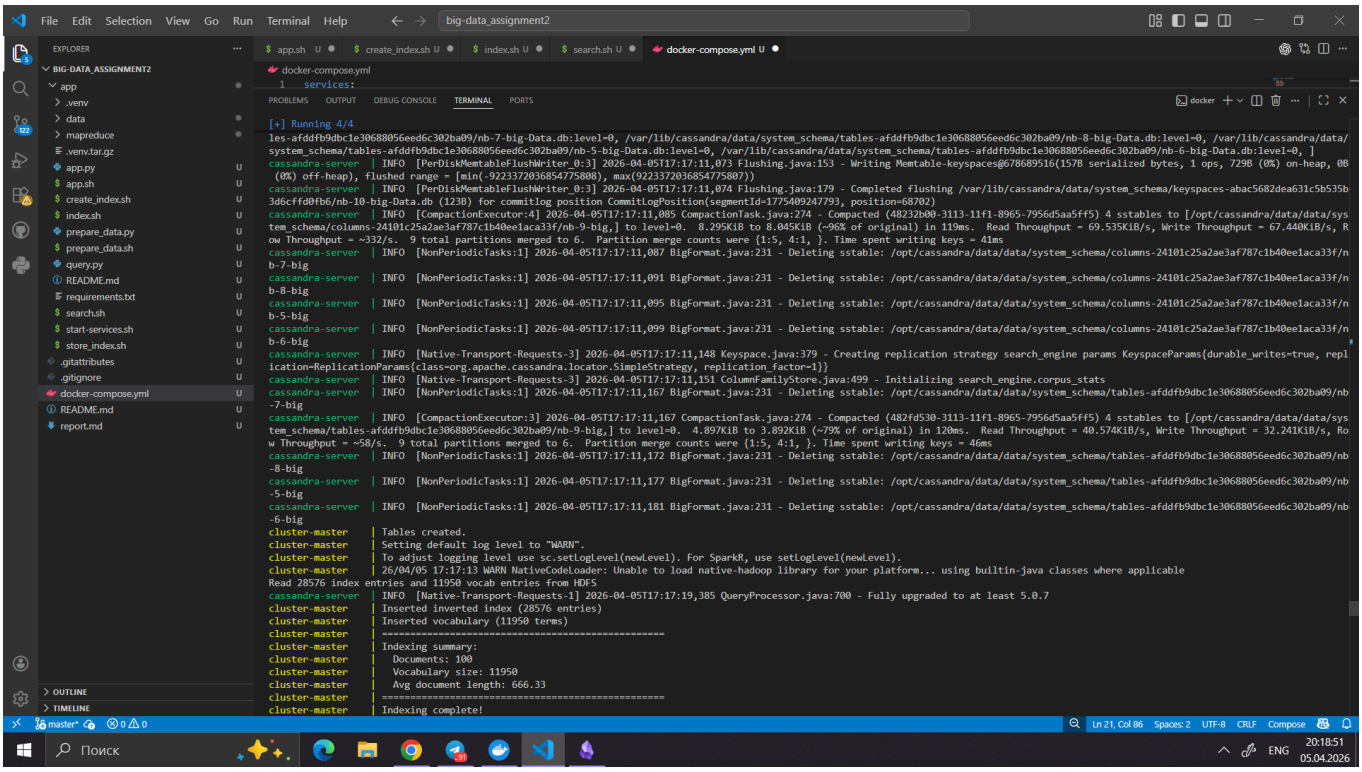
Indexing summary:

  Documents: 100

  Vocabulary size: 11950

  Avg document length: 666.33

==================================================
```



2.3 Search Queries

Three queries were executed to demonstrate the search engine. The results are shown below.

Query 1: "history of money"

```
Top 10 results for: history of money

------------------------------------------------------------

1. [Doc 10174562] A History of Money and Banking in the United States (score: 8.7067)

2. [Doc 14782494] A Countess Below Stairs (score: 4.1685)

3. [Doc 10399316] A Flowering Tree (score: 3.6363)

4. [Doc 10381993] A Doll's House (1973 Losey film) (score: 3.5891)

5. [Doc 13958799] A Dying Colonialism (score: 3.5388)

6. [Doc 1083442] A Hillbilly Tribute to ACDC (score: 3.5286)

7. [Doc 13060654] A Lion Among Men (score: 3.2905)

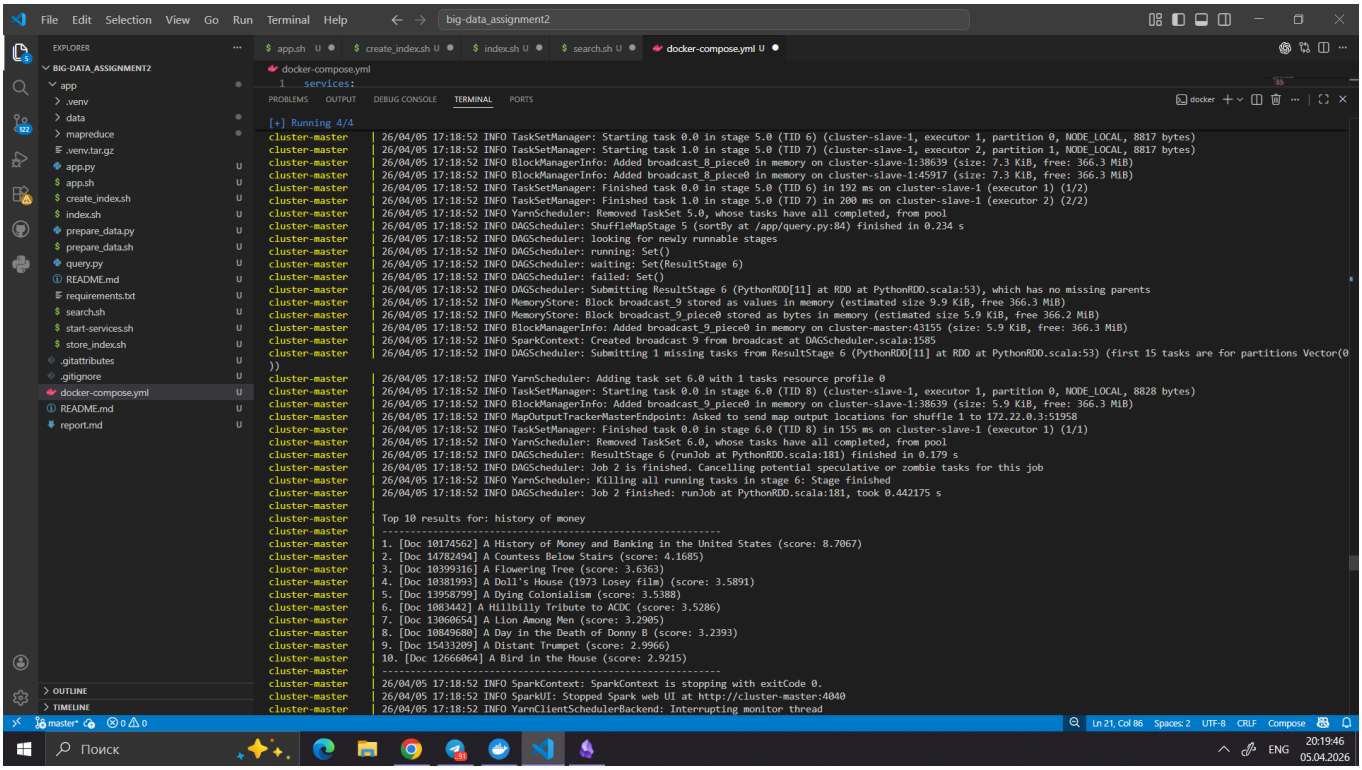
8. [Doc 10849680] A Day in the Death of Donny B (score: 3.2393)
```

```
9. [Doc 15433209] A Distant Trumpet (score: 2.9966)

10. [Doc 12666064] A Bird in the House (score: 2.9215)

------------------------------------------------------------
```

The top result, "A History of Money and Banking in the United States", is a direct and highly relevant match. Its BM25 score (8.71) is more than double the second result, which shows that the article contains the query terms with high frequency relative to its length. The remaining results contain some of the query terms ("history", "of", "money") distributed across their text, but with much lower relevance.



Query 2: "christmas story"

```
Top 10 results for: christmas story

------------------------------------------------------------

1. [Doc 12212399] A Flintstones Christmas Carol (score: 8.7705)

2. [Doc 12132506] A Crystal Christmas (score: 6.5054)

3. [Doc 12000397] A King and No King (score: 3.1876)
```

```
4. [Doc 1022877] A Death in the Family (comics) (score: 2.5723)

5. [Doc 10534798] A Black and White World (score: 2.2457)

6. [Doc 1472039] A Cinderella Story (soundtrack) (score: 2.1941)

7. [Doc 11528779] A Dreamer's Tales (score: 2.1189)

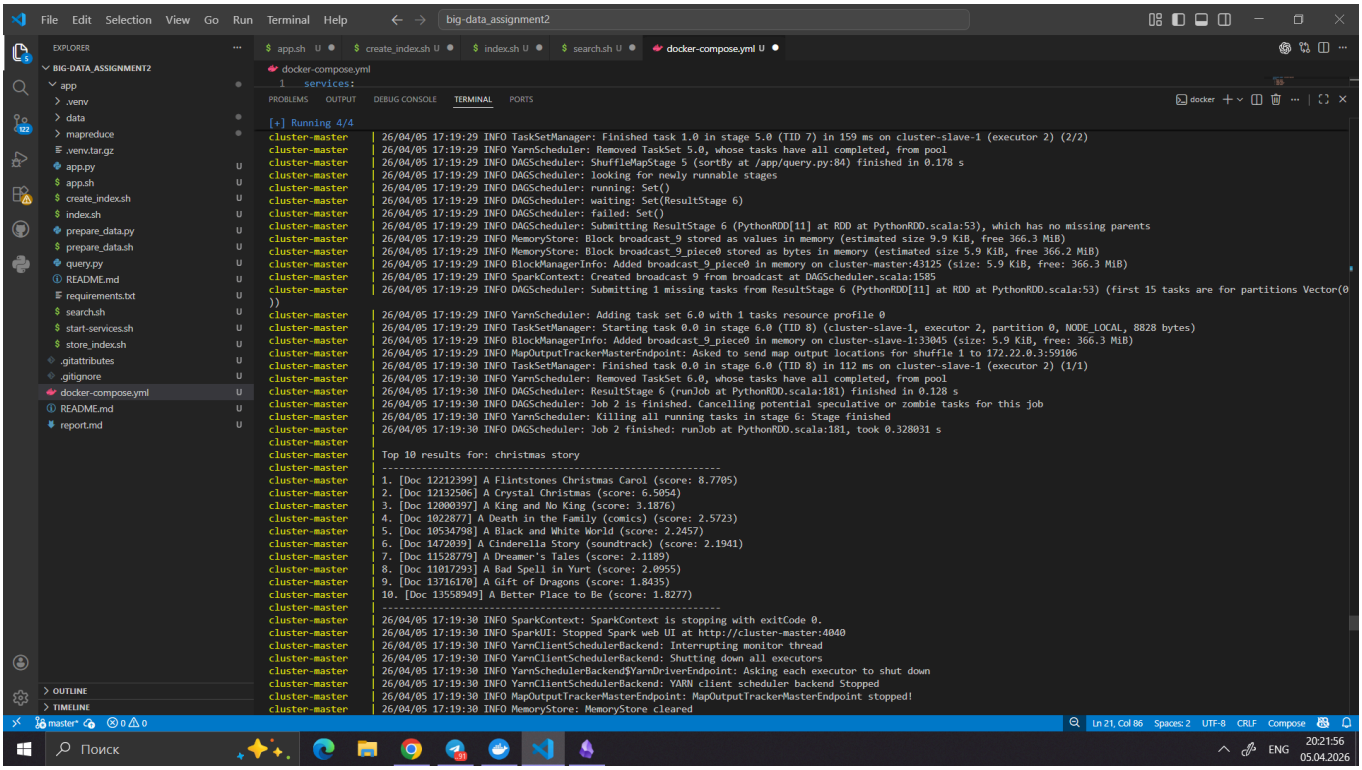
8. [Doc 11017293] A Bad Spell in Yurt (score: 2.0955)

9. [Doc 13716170] A Gift of Dragons (score: 1.8435)

10. [Doc 13558949] A Better Place to Be (score: 1.8277)

----------------------------------------------------------------
```

Both top results are Christmas-related articles. "A Flintstones Christmas Carol" ranks first because it is a Christmas-themed story, matching both query terms with high frequency. "A Crystal Christmas" ranks second as another Christmas album article. "A Cinderella Story (soundtrack)" at position 6 picks up the term "story" in its title and content. The results are consistent with what we would expect from a term-frequency-based ranking model.



Query 3: "death and war"

```
Top 10 results for: death and war

------------------------------------------------------------

1. [Doc 10031136] A Decade in the Grave (score: 6.5371)

2. [Doc 149791] A Double Life (1947 film) (score: 4.5897)

3. [Doc 15433209] A Distant Trumpet (score: 4.3600)

4. [Doc 14782494] A Countess Below Stairs (score: 4.3304)

5. [Doc 12666064] A Bird in the House (score: 4.2478)

6. [Doc 1022877] A Death in the Family (comics) (score: 4.0359)

7. [Doc 13958799] A Dying Colonialism (score: 3.9289)

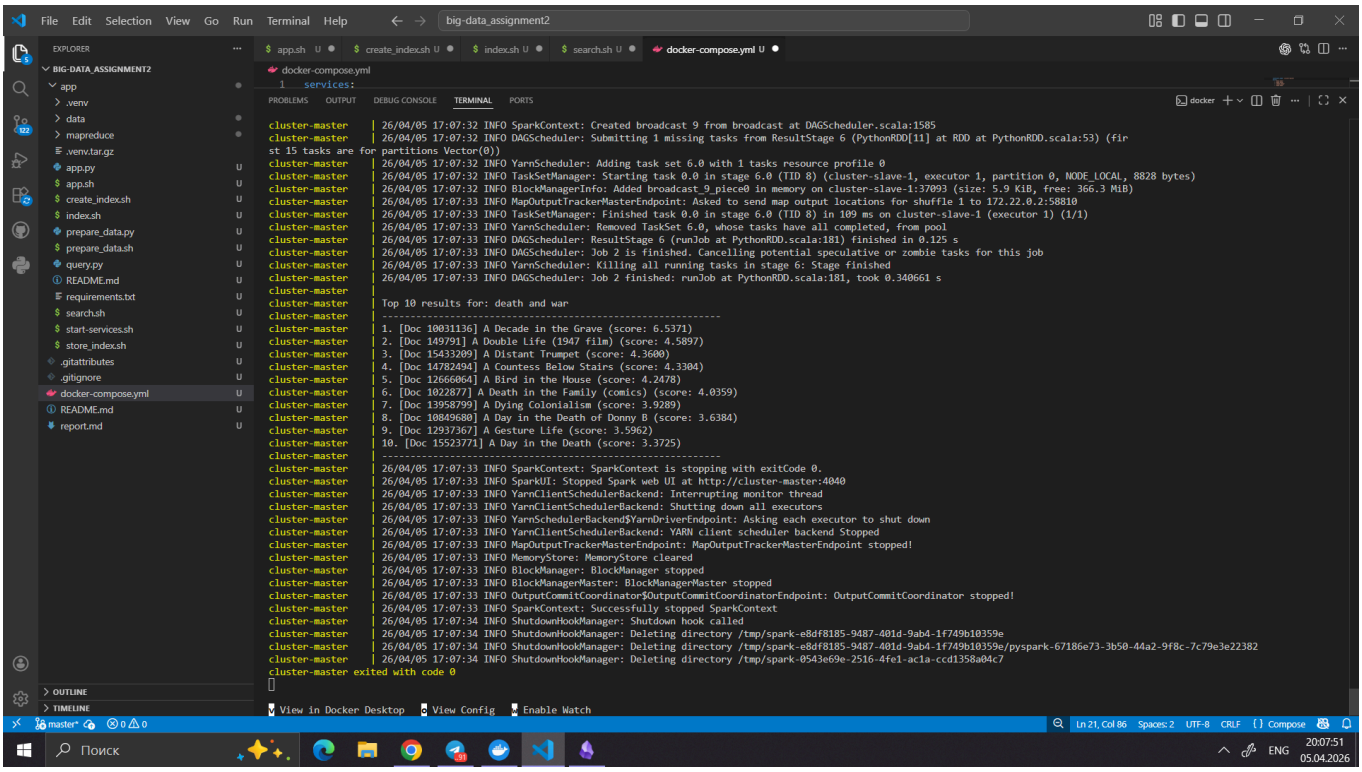
8. [Doc 10849680] A Day in the Death of Donny B (score: 3.6054)

9. [Doc 12937367] A Gesture Life (score: 3.5962)

10. [Doc 15523771] A Day in the Death (score: 3.3725)

------------------------------------------------------------
```

"A Decade in the Grave" is a compilation album article that references death-related themes extensively. "A Distant Trumpet" is a war film, matching the "war" term. "A Death in the Family (comics)" and "A Day in the Death" contain "death" directly. "A Dying Colonialism" by Frantz Fanon covers war and colonialism topics. The query term "and" is very common (appears in almost all documents), so its BM25 contribution is low due to the high df value, which is the correct behavior -- BM25 naturally downweights frequent terms through the IDF component.



2.4 Reflections

The BM25 model produces reasonable results for all three test queries. Documents whose titles directly match the query terms tend to rank highest, which is expected since the title words also appear in the document text (the text content includes the full article). The IDF weighting correctly reduces the influence of very common terms like "and", "of", and "the", while giving more weight to distinctive terms like "money", "christmas", or "war". The saturation parameter (k1 = 1.5) prevents documents that simply repeat a term many times from dominating the results, and the document length normalization (b = 0.75) ensures fair comparison between short and long articles.

References

- BM25 overview: https://en.wikipedia.org/wiki/Okapi_BM25
- TF-IDF and BM25 guide: https://www.ai-bites.net/tf-idf-and-bm25-for-rag-a-complete-guide/